```
"""
DCM Simulation — Task 3: Box-Counting Dimension Recovery
=========================================================
Computes effective dimension D_eff as a function of observer resolution R
using box-counting on the Mandelbrot set boundary.

Method:
- Generate Mandelbrot set at high resolution
- Apply box-counting at 12 resolution levels (coarse → fine)
- Record N(r) = number of boxes of side r needed to cover the boundary
- Compute D_eff(r) = log(N(r)) / log(1/r) at each level
- Compare to theoretical Mandelbrot fractal dimension D ≈ 2.0

Assumptions:
- Grid: 800x800 pixels, max 256 iterations
- Resolution levels: r = 2^k for k = 1..12 (coarse to fine)
- Boundary defined as pixels where |z| transitions across iteration threshold
"""

import numpy as np
import matplotlib.pyplot as plt

# ── Parameters ──────────────────────────────────────────────────────────
GRID_SIZE = 800
MAX_ITER = 256
N_LEVELS = 12
THEORETICAL_D = 2.0  # Mandelbrot boundary fractal dimension

plt.rcParams.update({'font.family': 'serif', 'font.size': 11})

# ── Generate Mandelbrot set ─────────────────────────────────────────────
def mandelbrot_grid(size, max_iter):
    x = np.linspace(-2.5, 1.0, size)
    y = np.linspace(-1.25, 1.25, size)
    C = x[np.newaxis, :] + 1j * y[:, np.newaxis]
    Z = np.zeros_like(C)
    M = np.zeros(C.shape, dtype=int)
    for i in range(max_iter):
        mask = np.abs(Z) <= 2
        Z[mask] = Z[mask]**2 + C[mask]
        M[mask] += 1
    return M

print("Generating Mandelbrot grid...")
```

```
M = mandelbrot_grid(GRID_SIZE, MAX_ITER)

# Boundary: pixels that escaped but are adjacent to non-escaped pixels
escaped = M < MAX_ITER
from scipy.ndimage import binary_dilation
boundary = escaped & ~binary_dilation(~escaped, iterations=2)
print(f"Boundary pixels: {boundary.sum()}")

# ── Box-counting at multiple resolution levels ──────────────────────
def box_count(binary_grid, box_size):
    """Count boxes of given size covering True pixels."""
    h, w = binary_grid.shape
    n_h = int(np.ceil(h / box_size))
    n_w = int(np.ceil(w / box_size))
    count = 0
    for i in range(n_h):
        for j in range(n_w):
            block = binary_grid[i*box_size:(i+1)*box_size,
                                j*box_size:(j+1)*box_size]
            if block.any():
                count += 1
    return count

box_sizes = [2**k for k in range(1, N_LEVELS + 1)]  # 2, 4, 8, ..., 4096
box_sizes = [b for b in box_sizes if b <= GRID_SIZE // 2]

print("Running box-counting...")
results = []
for bs in box_sizes:
    n = box_count(boundary, bs)
    r = bs / GRID_SIZE  # normalized resolution
    if n > 1 and r < 1:
        D_eff = np.log(n) / np.log(1.0 / r)
        results.append((r, bs, n, D_eff))
        print(f"  box_size={bs:4d}, r={r:.4f}, N={n:6d}, D_eff={D_eff:.4f}")

results = np.array(results)
r_vals = results[:, 0]
D_eff_vals = results[:, 3]

# ── Plot ────────────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 5.5))
fig.suptitle("DCM — Figure 2: Box-Counting Resolution Curve (Mandelbrot Set)\n"
             "Effective dimension D_eff as a function of observer resolution R",
             fontsize=12, fontweight='bold')

ax.plot(r_vals, D_eff_vals, 'o-', color='#2E5596', lw=2, ms=7,
```

```
        label='$D_{eff}$ (box-counting)')
ax.axhline(THEORETICAL_D, color='#C00000', ls='--', lw=1.5,
           label=f'Theoretical $D \\approx {THEORETICAL_D}$ (Mandelbrot boundary)

# Annotate direction
ax.annotate('Coarse resolution\n(compression)', xy=(r_vals[-1], D_eff_vals[-1]),
            xytext=(r_vals[-1]*0.6, D_eff_vals[-1]-0.3),
            fontsize=9, color='gray',
            arrowprops=dict(arrowstyle='->', color='gray'))
ax.annotate('Fine resolution\n(recovery)', xy=(r_vals[0], D_eff_vals[0]),
            xytext=(r_vals[0]*3, D_eff_vals[0]-0.25),
            fontsize=9, color='#2E5596',
            arrowprops=dict(arrowstyle='->', color='#2E5596'))

ax.set_xscale('log')
ax.invert_xaxis()
ax.set_xlabel('Observer Resolution R (normalized, log scale; fine → coarse →)', f
ax.set_ylabel('Effective Dimension $D_{eff}$', fontsize=10)
ax.set_ylim(0.5, 2.5)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/home/claude/figure2_boxcounting.png', dpi=150, bbox_inches='tight')
plt.close()

# Save data
np.savetxt('/home/claude/task3_boxcounting_data.csv',
           np.column_stack([r_vals, results[:,1], results[:,2], D_eff_vals]),
           header='resolution_r,box_size_px,box_count_N,D_eff',
           delimiter=',', comments='')

print("\nTask 3 complete. Figure and data saved.")
print(f"D_eff range: {D_eff_vals.min():.3f} – {D_eff_vals.max():.3f}")
print(f"Theoretical: {THEORETICAL_D}")
```